

Dossier de Projet — Reminder Famille

Titre Professionnel : Développeur Web et Web Mobile (DWWM) **Centre de formation** : AFPA Marseille

Candidat : Vladimir Rodzaevskiy **Année de soutenance** : 2026

Sommaire

1. [Introduction générale](#)
 2. [Présentation du projet](#)
 3. [Équipe et gestion de projet](#)
 4. [Spécifications fonctionnelles](#)
 5. [Spécifications techniques](#)
 6. [Maquettage](#)
 7. [Intégration](#)
 8. [Conception de la base de données](#)
 9. [Mise en place de la base de données](#)
 10. [Mission n°1 — Authentification & Familles \(CRUD\)](#)
 11. [Mission n°2 — Tâches & Validation parent \(AJAX\)](#)
 12. [Mission n°3 — Export iCal \(intégration standard tiers\)](#)
 13. [Déploiement](#)
 14. [Sécurité](#)
 15. [Conclusion générale](#)
 16. [Annexes](#)
-

1. Introduction générale

Reminder Famille est une application web full-stack permettant à une famille de gérer collectivement ses tâches récurrentes : rendez-vous, factures, démarches administratives, corvées domestiques. L'application introduit un modèle parent/enfant unique sur le marché et synchronise les échéances avec le calendrier natif du téléphone — sans nécessiter l'installation d'une application mobile.

Ce dossier présente la conception, l'architecture et l'implémentation du projet, ainsi que les choix techniques et leur justification dans le cadre du Titre Professionnel **Développeur Web et Web Mobile**.

 **Code source** : <https://github.com/Vladimir08880888/reminder-famille>  **Dossiers AFPA** : <https://github.com/Vladimir08880888/vladimir-rodzaevski-dossiers-pros-DWWM>

2. Présentation du projet

2.1 Contexte

Une famille moderne gère un volume important de tâches récurrentes :

- Échéances financières (loyer, assurances, factures)
- Rendez-vous médicaux (dentiste, vaccins, contrôles)
- Démarches administratives (carte d'identité, impôts, certificats)
- Maintenance véhicule (contrôle technique, vidange, assurance)
- Corvées domestiques (poubelle, courses, ménage hebdo)
- Suivi scolaire des enfants (devoirs, projets, événements)

Ces obligations sont aujourd'hui dispersées entre :

- Post-it sur le frigo
- Calendriers personnels non partagés
- Conversations WhatsApp familiales (perdues dans le scroll)
- Mémoire d'un seul parent (charge mentale)

2.2 Analyse du marché

Les solutions existantes présentent toutes des limitations pour l'usage familial :

Solution	Limitation
Todoist	Pensé entreprise, fonctions famille payantes (4€/mois/user)
Trello	Aucun modèle parent/enfant, complexe pour non-tech
Microsoft Family	Lock-in écosystème Microsoft
Apple Reminders	iOS uniquement
Google Tasks	Pas de partage à plusieurs
Notion	Trop complexe, abonnement, pas de notifications natives

2.3 Besoin client identifié

Une famille typique a besoin de :

1. Une **vision partagée** des obligations de chacun
2. Un système de **délégation** parent → enfant avec **validation**
3. Des **rappels fiables** sur **tous les téléphones** (iOS + Android)
4. **Aucun coût** récurrent ni installation d'application
5. Une distinction claire entre **tâches perso** et **tâches familiales**

2.4 Hypothèse de solution

Construire une application web qui :

- Modélise nativement le concept de **famille** avec rôles parent/enfant
- Implémente un **workflow de validation** quand l'enfant termine une tâche
- Exploite le **standard ouvert iCal** (RFC 5545) plutôt que des notifications propriétaires
- Reste gratuite et auto-hébergeable

2.5 Historique du projet

Date	Étape
2026-04-21	Rédaction du Cahier des Charges (v1)
2026-04-25	Première itération : application solo (sans familles)
2026-05-10	Pivot vers modèle multi-utilisateurs avec familles
2026-05-19	CDC v4 finalisé, skeleton du projet créé
2026-05-22	Ajout du flow validation parent
2026-05-25	Ajout calendrier mensuel, stats, commentaires

2026-05-27	Multi-canaux iCal (perso + par famille)
2026-05-28	Suite de 57 tests d'intégration, dossiers AFPA
2026-06-XX	Soutenance

3. Équipe et gestion de projet

3.1 Composition de l'équipe

Projet **solo**, réalisé par Vladimir Rodzaevskiy (candidat DWWM).

```
Vladimir Rodzaevskiy
- Conception & cadrage
- Développement back (Node/Express)
- Développement front (React)
- Base de données (MariaDB)
- Tests d'intégration
- Documentation
```

3.2 Méthodologie choisie

Approche Agile allégée (solo). Pas de scrum formel mais :

- **Sprints d'1 semaine** avec un objectif clair (ex: « semaine 1 = auth »)
- **Itération incrémentale** : chaque feature livrée fonctionnelle bout en bout
- **Tests continus** : chaque endpoint testé manuellement (`curl`) puis par script d'intégration
- **Refactoring permanent** quand un pattern se répète 3 fois

3.3 Outils utilisés

Domaine	Outil
Versioning	Git (local) + GitHub
Suivi de tâches	TODO en markdown dans le repo + commits descriptifs
Communication	N/A (solo)
IDE	Visual Studio Code
Tests	<code>curl</code> , script bash <code>tests/integration.sh</code>
BDD locale	MariaDB sur Fedora 43
Documentation	Markdown versionné, rendu natif GitHub

3.4 Planning réel (8 semaines)

Sem.	Objectif	Statut
1	CDC + skeleton projet + BDD initiale	✓

2	API auth (register, login, JWT)	✓
3	API familles (create, join, approve)	✓
4	API tâches + récurrence	✓
5	Front auth + AuthContext + ProtectedRoute	✓
6	Front familles + tâches + assignation	✓
7	Dashboards parent/child + iCal export	✓
8	Tests, corrections, documentation, soutenance	✓

4. Spécifications fonctionnelles

4.1 Diagramme de packages

Voir [diagrammes/packages.mmd](#) (UML).

Architecture en **deux applications autonomes** communiquant via HTTP/JSON :

- **Front** (React/Vite) : pages → composants → contextes + appels API
- **Back** (Express) : routes → middleware → controllers → models → DB

4.2 Diagramme de cas d'utilisation

Voir [diagrammes/use-cases.mmd](#) (UML).

6 acteurs identifiés :

- Visiteur (non authentifié)
- Membre (utilisateur authentifié)
- Parent (rôle dans famille)
- Enfant (rôle dans famille)
- Admin famille (parent + is_admin)
- Calendrier externe (iCal consumer)

28 cas d'utilisation couvrant l'inscription, la gestion de familles, la gestion de tâches, la validation parent, et la synchronisation iCal.

4.3 User stories

Voir [user-stories.md](#) — **23 user stories** au format Connextra, regroupées par domaine (auth, familles, tâches perso, tâches familiales, visualisation, notifications, confort).

4.4 Compréhension sans le code

L'application repose sur des concepts simples du quotidien :

Marie crée une famille. Elle reçoit un code à partager. Paul rejoint la famille avec le code. Marie le valide en lui attribuant le rôle « parent ». Léo rejoint à son tour, Marie le valide comme « enfant ».

Marie assigne à Léo la tâche « Sortir la poubelle » chaque lundi. Léo voit la tâche sur son dashboard. Quand il l'a faite, il clique « Terminer ». Marie reçoit une notification : « Léo dit avoir terminé Sortir la poubelle. » Elle clique « Valider ». La tâche réapparaît automatiquement la semaine suivante.

Marie scanne le QR code de son profil avec son iPhone. Désormais ses échéances apparaissent dans son calendrier natif avec rappel la veille.

5. Spécifications techniques

5.1 Stack technique

Couche	Technologie	Version	Justification
Runtime	Node.js	22 LTS	Stable, support long terme, perf V8
API	Express	4.19	Mature, minimaliste, écosystème massif
Driver BDD	mysql2	3.11	Prepared statements natifs, promesses
Auth	jsonwebtoken	9.0	Standard JWT, HS256
Hash	bcrypt	5.1	Algorithme adapté aux mots de passe
iCal	ical-generator	7.2	Génération RFC 5545 + VALARM
Rate limit	express-rate-limit	7.5	Anti brute-force sur /auth
Front	React	18.3	Composants, écosystème, mémo automatique
Bundler	Vite	5.4	HMR ultra-rapide, build ESM natif
Routing	React Router	6.26	Standard de fait React
Charts	Chart.js + react-chartjs-2	4.5 / 5.3	Léger, customisable, accessible
Icons	lucide-react	1.16	Outline cohérent, tree-shakeable
QR	qrcode	1.5	Génération canvas/SVG pure JS
BDD	MariaDB	10.11	Open source, ACID, FK strictes, JSON support

5.2 Installation de l'environnement

Gestionnaire de dépendances : npm

```
# Back
cd back && npm install
# Front
cd front && npm install
```

Git (workflow)

- **Branche unique** `main` (solo, pas de PR)
- **Commits atomiques** avec message en français descriptif
- `.gitignore` couvre `node_modules/` , `.env` , `dist/` , `.DS_Store`

Variables d'environnement (`.env`)

```
PORT=3000
NODE_ENV=development
FRONT_ORIGIN=http://localhost:5173

DB_HOST=localhost
DB_PORT=3306
DB_USER=reminder
DB_PASSWORD=changeme
DB_NAME=reminder_db

JWT_SECRET=<32 caractères aléatoires>
JWT_EXPIRES_IN=7d
```

Un `.env.example` versionné documente les variables attendues.

Pas de Docker

Choix assumé : développement local direct sur Fedora 43. Docker prévu pour le déploiement production (voir section 13).

5.3 Adaptation à l'environnement

L'application gère **trois environnements** via `NODE_ENV` :

Env	NODE_ENV	Comportement
Développement	development	Verbose logs, rate limit large, CORS souple
Staging	staging	Logs réduits, rate limit prod, CORS strict
Production	production	Aucun log debug, rate limit strict, HTTPS forcé

6. Maquettage

6.1 Charte graphique

Voir [maquettes/charte-graphique.md](#) — document complet (couleurs primaires, statuts, priorités, catégories, typographies, spacing, animations).

Identité visuelle :

- Dégradé indigo → violet → rose pour la marque
- Glassmorphism (translucidité + flou)
- Mode sombre intégré

6.2 Typographies

- **UI** : *system font stack* avec Inter en priorité après les fonts système
- **Code/IDs** : JetBrains Mono → fallbacks mono système

Justification : zéro téléchargement de font, performance maximale, rendu natif par chaque OS.

6.3 Wireframes

Voir [maquettes/wireframes.md](#) — wireframes ASCII des 7 écrans clés (login, dashboards parent/child, liste tâches, formulaire, détail famille, profil iCal).

6.4 Mockups haute fidélité

Les **17 screenshots** définitifs sont disponibles dans le repo projet : [reminder-famille/annexes/screenshots/](#)

Ils ont été automatisés via un script **Playwright** garantissant des captures cohérentes et facilement régénérables.

6.5 Choix UX/UI justifiés

Choix	Justification
Dashboard dispatch parent/enfant	Le contenu pertinent diffère radicalement selon le rôle
Filtres en haut de la liste de tâches	Réduit la surcharge cognitive sur petits écrans
Toast top-right pour notifications	Standard adopté par Material, iOS — pas de surprise
Modal de confirmation pour suppression	Évite les destructions accidentelles
QR code pour iCal	Plus rapide que copier-coller une URL longue
Couleurs par catégorie	Identification visuelle instantanée (Hick's law)

6.6 Versions desktop / tablette / mobile

Tous les écrans sont pensés **mobile-first** (375px) puis enrichis :

```
/* Approche mobile-first dans variables.css */
/* Par défaut : mobile */
.container { padding: 1rem; }

@media (min-width: 768px) { /* tablette */
  .container { padding: 2rem; max-width: 720px; }
}
@media (min-width: 1024px) { /* desktop */
  .container { max-width: 1180px; }
}
```

7. Intégration

7.1 Responsive design

Mobile (< 768px)

- Navbar → menu burger (icône hamburger)
- Cartes pleine largeur, espacement réduit
- Tableaux remplacés par cartes empilées
- Cibles tactiles ≥ 44px (Apple HIG)

Tablette (768–1024px)

- Navbar visible avec moins d'éléments
- 2 colonnes pour cartes statistiques

Desktop (≥ 1024px)

- Navbar complète, sidebar fixe pour navigation
- 3-4 colonnes pour stats
- Survol enrichi (hover states)

7.2 Media queries

```
/* Exemple — front/src/styles/index.css */
.dashboard-grid {
  display: grid;
  gap: 1rem;
  grid-template-columns: 1fr; /* mobile : 1 colonne */
}

@media (min-width: 768px) {
  .dashboard-grid {
    grid-template-columns: repeat(2, 1fr); /* tablette : 2 col */
  }
}

@media (min-width: 1024px) {
  .dashboard-grid {
    grid-template-columns: repeat(4, 1fr); /* desktop : 4 col */
  }
}
```

7.3 Extraits HTML/CSS pertinents

Glassmorphism — composant carte

```
.glass-card {
  background: var(--glass);
  backdrop-filter: var(--blur-md);
  -webkit-backdrop-filter: var(--blur-md);
  border: 1px solid var(--glass-border);
  border-radius: var(--r-lg);
  box-shadow: var(--shadow-md);
  padding: 1.5rem;
  transition: transform var(--t-base) var(--ease),
    box-shadow var(--t-base) var(--ease);
}

.glass-card:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-lg);
}
```

Bouton primaire avec dégradé


```

.btn-primary {
  background: var(--primary-grad);
  color: var(--text-on-primary);
  border: none;
  padding: 0.75rem 1.5rem;
  border-radius: var(--r-sm);
  font-weight: 600;
  cursor: pointer;
  transition: transform var(--t-fast) var(--ease);
}

.btn-primary:hover {
  background: var(--primary-grad-hover);
  transform: scale(1.02);
}

.btn-primary:focus-visible {
  outline: none;
  box-shadow: var(--shadow-glow);
}

```

7.4 Accessibilité

- **Focus visible** sur tous les éléments interactifs (`:focus-visible`)
- **prefers-reduced-motion** respecté (animations désactivées)
- **Contraste WCAG AA** vérifié (ratio ≥ 4.5 pour le texte)
- **Labels associés** à tous les inputs (`<label htmlFor>`)
- **alt sur toutes les images**

8. Conception de la base de données

8.1 MCD (Merise — Modèle Conceptuel)

Voir [diagrammes/mcd.mmd](#) .

5 entités :

- USER (utilisateur)
- FAMILY (famille)
- FAMILY_MEMBER (appartenance avec rôle)
- TASK (tâche)
- TASK_COMMENT (commentaire)

8.2 MLD (Merise — Modèle Logique)

Voir [diagrammes/mlld.mmd](#) .

Les relations many-to-many sont matérialisées par la table d'association `family_members` avec une **clé primaire composite** (`family_id`, `user_id`) .

8.3 MPD (Merise — Modèle Physique)

Les fichiers SQL `CREATE TABLE` sont versionnés dans le repo du projet : [back/migrations/*.sql](#) (6 fichiers).

8.4 Cardinalités

Relation	Cardinalités	Sens
User crée Family	1,1 → 0,N	Un user peut créer 0+ familles ; chaque famille a 1 créateur
User appartient à Family (via Family_Member)	0,N ↔ 0,N	Many-to-many
User crée Task	1,1 → 0,N	Une tâche a 1 auteur
Family contient Task	0,1 → 0,N	Tâche perso si family_id NULL
User assigné à Task	0,1 → 0,N	Optionnel
Task reçoit Comment	1,1 → 0,N	Un comment appartient à 1 tâche

8.5 Propriétés ACID

MariaDB (moteur **InnoDB**) garantit les propriétés ACID :

Propriété	Garantie dans le projet
A tomacité	Une INSERT échoue intégralement si une FK est violée
C ohérence	Les FK ON DELETE CASCADE/SET NULL maintiennent l'intégrité
I solation	Niveau REPEATABLE READ par défaut → pas de lectures sales
D urabilité	InnoDB persiste sur disque avec WAL (write-ahead log)

8.6 Choix technologique : MariaDB vs PostgreSQL vs NoSQL

Critère	MariaDB	PostgreSQL	MongoDB
ACID strict	✓	✓	⚠ depuis 4.0
FK natives	✓	✓	✗
Open source	✓ (GPL)	✓ (PostgreSQL)	✓ (SSPL — copyleft)
Compatibilité MySQL	✓ totale	✗	✗
Écosystème JS	✓✓ (mysql2 perf top)	✓ (pg)	✓ (mongoose)
Pour ce projet	Adapté	Adapté	Inadapté (relations strictes)

Choix : MariaDB pour son écosystème JS mature, sa compatibilité MySQL (documentation pléthorique), et car le projet de référence proposé en formation l'utilisait également.

8.7 Type de famille NoSQL : non applicable

Le projet repose sur des **relations strictes** (FK obligatoires entre users, families, family_members, tasks). Un NoSQL document store demanderait une dénormalisation contre-productive et perdrait la garantie d'intégrité référentielle.

9. Mise en place de la base de données

9.1 Création de la BDD

```
CREATE DATABASE reminder_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'reminder'@'localhost'
  IDENTIFIED BY '<mdp local>';

GRANT ALL PRIVILEGES ON reminder_db.*
  TO 'reminder'@'localhost';

FLUSH PRIVILEGES;
```

9.2 Système de migrations

Toutes les modifications de schéma sont des **migrations versionnées** dans `back/migrations/*.sql`. Un runner (`back/src/db/migrate.js`) maintient une table `_migrations(filename, applied_at)` pour exécuter uniquement les fichiers non encore appliqués.

9.3 CREATE TABLE — exemple représentatif

```
-- back/migrations/004_tasks.sql
CREATE TABLE IF NOT EXISTS tasks (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  family_id   INT DEFAULT NULL,
  assigned_to INT DEFAULT NULL,
  title       VARCHAR(200) NOT NULL,
  description  TEXT          DEFAULT NULL,
  category     ENUM('Santé', 'Finances', 'Administratif', 'Véhicule',
                    'Logement', 'Corvée', 'Autre') NOT NULL DEFAULT 'Autre',
  priority     ENUM('low', 'medium', 'high') NOT NULL DEFAULT 'medium',
  frequency    ENUM('once', 'daily', 'weekly', 'monthly', 'yearly') NOT NULL DEFAULT
'once',
  due_date     DATE NOT NULL,
  status       ENUM('pending', 'completed') NOT NULL DEFAULT 'pending',
  completed_at DATETIME DEFAULT NULL,
  completed_by INT          DEFAULT NULL,
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
```

```

FOREIGN KEY (family_id) REFERENCES families(id) ON DELETE CASCADE,
FOREIGN KEY (assigned_to) REFERENCES users(id) ON DELETE SET NULL,
FOREIGN KEY (completed_by) REFERENCES users(id) ON DELETE SET NULL,

INDEX idx_tasks_user (user_id),
INDEX idx_tasks_family (family_id),
INDEX idx_tasks_assigned (assigned_to),
INDEX idx_tasks_due_date (due_date),
INDEX idx_tasks_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

9.4 ALTER TABLE — exemple

Ajout du statut `pending_review` après ajout du flow de validation parent :

```

-- back/migrations/005_pending_review.sql
ALTER TABLE tasks
  MODIFY COLUMN status
    ENUM('pending', 'pending_review', 'completed')
  NOT NULL DEFAULT 'pending';

```

9.5 FOREIGN KEY et stratégies de suppression

Relation	ON DELETE	Pourquoi
tasks.user_id → users.id	CASCADE	Si user supprimé, ses tâches perso disparaissent
tasks.family_id → families.id	CASCADE	Famille dissoute = tâches familiales supprimées
tasks.assigned_to → users.id	SET NULL	User parti, tâche reste mais sans assigné
tasks.completed_by → users.id	SET NULL	Historique préservé même si user supprimé
family_members.family_id → families.id	CASCADE	Famille dissoute = membres détachés
family_members.user_id → users.id	CASCADE	User supprimé = retiré de toutes les familles
families.created_by → users.id	RESTRICT	Empêche suppression user si famille active

9.6 Connexion BDD selon environnement

Configuration centralisée dans `back/src/config/env.js` qui lit `.env` :

- `DB_HOST` , `DB_PORT` , `DB_USER` , `DB_PASSWORD` , `DB_NAME`
- Pool de connexions (10 max par défaut) géré par `mysql2/promise`

Aucune URL ou credential codé en dur dans le code source.

10. Mission n°1 — Authentification & Familles (CRUD)

10.1 Introduction

Cette mission couvre la **gestion des utilisateurs et des familles** : inscription, connexion, création de famille, jointure par code, validation des membres, gestion des rôles.

C'est le **socle de toute l'application** — sans ces fonctionnalités, le reste n'a aucun sens.

10.2 Tables concernées

- `users` (id, email, password_hash, first_name, last_name, calendar_token)
- `familles` (id, name, invite_code, created_by)
- `family_members` (family_id, user_id, role, is_admin, status)

10.3 Use cases spécifiques

- US-01 — Création de compte
- US-02 — Connexion
- US-03 — Changement de mot de passe
- US-04 — Création d'une famille
- US-05 — Rejoindre une famille via code
- US-06 — Validation des nouveaux membres
- US-07 — Régénération du code d'invitation
- US-08 — Réinitialisation MDP d'un membre
- US-09 — Quitter une famille

10.4 Diagramme d'activité — Authentification

Voir [diagrammes/activite-auth.mmd](#) .

10.5 Diagramme de séquence — CRUD tâche familiale

Voir [diagrammes/sequence-task-crud.mmd](#) .

10.6 Architecture MVC

```
HTTP Request
→ Route (mapping URL → handler)
→ Middleware (authRequired, requireFamilyMember...)
→ Controller (logique métier)
→ Validator (entrée HTTP)
→ Model (requête SQL préparée)
→ MariaDB
← retour ligne(s)
← réponse JSON
← réponse HTTP
```

10.7 Validation des données

Côté front

HTML5 (`required` , `type="email"` , `minlength`) + retours d'erreurs API affichés sous chaque champ via state React.

Côté back (sécurité réelle)

```
// back/src/validators/auth.validator.js
export function validateRegister(body) {
  const errors = {};
  const { email, password, first_name, last_name } = body || {};

  if (!email || !EMAIL_RE.test(email) || email.length > 255)
    errors.email = 'Email invalide';
  if (!password || password.length < 8)
    errors.password = 'Mot de passe d\'au moins 8 caractères';
  if (!first_name || first_name.length > 100)
    errors.first_name = 'Prénom requis';
  if (!last_name || last_name.length > 100)
    errors.last_name = 'Nom requis';

  if (Object.keys(errors).length)
    throw badRequest('Champs invalides', errors);
  return { email, password, first_name, last_name };
}
```

10.8 Formulaire HTML de qualité

```
// front/src/pages/Register.jsx (extrait)
<form onSubmit={handleSubmit} noValidate>
  <label htmlFor="email">Email</label>
  <input
    id="email"
    type="email"
    autoComplete="email"
    required
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    aria-invalid={!errors.email}
    aria-describedby={errors.email ? 'email-error' : undefined}
  />
  {errors.email && (
    <span id="email-error" className="error">{errors.email}</span>
  )}
  { /* ... */ }
</form>
```

Bonnes pratiques :

- `<label htmlFor>` lié à `<input id>`
- `autoComplete` pour suggestions navigateur
- `aria-invalid` + `aria-describedby` pour lecteurs d'écran
- `noValidate` car validation gérée par React (UX cohérente)

10.9 Code back — Routes

```
// back/src/routes/auth.routes.js
import { Router } from 'express';
import { authController } from '../controllers/auth.controller.js';
import { authRequired } from '../middleware/auth.js';
import { loginLimiter, registerLimiter } from '../middleware/rateLimit.js';
import { asyncHandler } from '../utils/asyncHandler.js';

export const router = Router();

router.post('/register', registerLimiter, asyncHandler(authController.register));
router.post('/login', loginLimiter, asyncHandler(authController.login));
router.get('/me', authRequired, asyncHandler(authController.me));
router.post('/change-password', authRequired,
  asyncHandler(authController.changePassword));
```

10.10 Contrôleurs CRUD

back/src/controllers/families.controller.js expose les actions :

- list (lire)
- create (créer)
- detail (lire un)
- rename (modifier)
- remove (supprimer)
- join , approve , updateMember , removeMember , regenerateCode , leave , resetMemberPassword

10.11 Gestion des rôles

Action	parent	child	admin
Voir tâches familiales	✓	✓ (assignées)	✓
Créer tâche familiale	✓	✗	✓
Modifier/supprimer tâche	✓	✗	✓
Compléter tâche assignée	✓	✓ → pending_review	✓
Valider pending_review	✓	✗	✓
Approuver membre	✗	✗	✓
Régénérer code	✗	✗	✓
Reset MDP autre membre	✗	✗	✓

10.12 Authentification

JWT (HS256) signé avec JWT_SECRET , expiration 7 jours :

```
// back/src/services/jwt.service.js
import jwt from 'jsonwebtoken';
```

```
import { env } from '../config/env.js';

export function signToken(userId) {
  return jwt.sign({ sub: userId }, env.jwt.secret, {
    expiresIn: env.jwt.expiresIn,
    algorithm: 'HS256',
  });
}

export function verifyToken(token) {
  return jwt.verify(token, env.jwt.secret);
}
```

Stockage côté front : **localStorage** sous la clé `reminder_token` . Inclus automatiquement dans les requêtes via le wrapper `api/client.js` .

10.13 Modèle — SQL préparé

```
// back/src/models/user.model.js
export const userModel = {
  async create({ email, password_hash, first_name, last_name, calendar_token }) {
    const [r] = await pool.query(
      `INSERT INTO users (email, password_hash, first_name, last_name,
calendar_token)
VALUES (?, ?, ?, ?, ?)`,
      [email, password_hash, first_name, last_name, calendar_token]
    );
    return r.insertId;
  },

  async findByEmail(email) {
    const [rows] = await pool.query(
      'SELECT * FROM users WHERE email = ?',
      [email]
    );
    return rows[0] || null;
  },
  // ...
};
```

10.14 Mini-bilan Mission 1

✅ Toutes les user stories de cette mission sont livrées et testées (28/57 scénarios d'intégration concernent l'auth et les familles).

Difficultés rencontrées :

- **Last-admin protection** — il a fallu vérifier ce cas à 2 endroits (`leave` + `removeMember`)
- **Distinction « pas membre » vs « membre pending »** — un user `pending` reste membre techniquement mais ne doit voir aucune donnée

11. Mission n°2 — Tâches & Validation parent (AJAX)

11.1 Introduction

Cette mission couvre le **cœur métier** de l'application : la création, la consultation, la complétion des tâches, et surtout le **workflow de validation parent** qui distingue Reminder Famille des autres todos.

Toutes les interactions sont **asynchrones** (AJAX via `fetch`) — la page n'est jamais rechargée.

11.2 API et AJAX

Le front communique avec le back via `fetch` (API native moderne) wrappé dans un client unifié :

```
// front/src/api/client.js
export async function apiRequest(path, options = {}) {
  const token = localStorage.getItem('reminder_token');
  const res = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token && { Authorization: `Bearer ${token}` }),
      ...options.headers,
    },
  });
  const body = await res.json();
  if (!res.ok) throw new ApiError(body, res.status);
  return body;
}
```

11.3 Diagramme d'activité — Validation parent

Voir [diagrammes/activite-validation-parent.mmd](#) .

11.4 Diagramme de séquence — Création tâche

Voir [diagrammes/sequence-task-crud.mmd](#) .

11.5 Documentation API (style Swagger)

Voir [api-documentation.md](#) — **30+ endpoints** documentés avec payload, réponses, codes de statut.

11.6 Code JS asynchrone (fetch + promises)

```
// front/src/api/tasks.api.js
import { apiRequest } from './client.js';

export const tasksApi = {
  list: (filters = {}) => apiRequest(`/tasks?${new URLSearchParams(filters)}`),
  create: (data) => apiRequest('/tasks', { method: 'POST', body:
JSON.stringify(data) }),
  update: (id, data) => apiRequest(`/tasks/${id}`, { method: 'PUT', body:
JSON.stringify(data) }),
}
```

```

remove:    (id) => apiRequest(`/tasks/${id}`, { method: 'DELETE' }),
complete:  (id) => apiRequest(`/tasks/${id}/complete`, { method: 'POST' }),
approve:   (id) => apiRequest(`/tasks/${id}/approve`, { method: 'POST' }),
reject:    (id) => apiRequest(`/tasks/${id}/reject`, { method: 'POST' }),
};

```

11.7 Stockage des données

Donnée	Stockage	Raison
Tâches, familles, users	MariaDB	Source de vérité
JWT token	localStorage	Persistant entre sessions navigateur
Préférence thème	localStorage	UX immédiate au reload
Famille active sélectionnée	localStorage	UX cohérente
État UI temporaire	React state (useState)	Volatile

11.8 Format JSON

Tout le trafic API est en **JSON** (Content-Type: application/json).

Exemple réponse :

```

{
  "id": 12,
  "title": "Sortir poubelle",
  "category": "Corvée",
  "priority": "medium",
  "frequency": "weekly",
  "due_date": "2026-05-29",
  "status": "pending"
}

```

Exemple erreur :

```

{
  "error": "Champs invalides",
  "details": {
    "title": "Titre invalide (1-200 caractères)",
    "due_date": "Date invalide (YYYY-MM-DD)"
  }
}

```

11.9 Outils de test API

Outil	Usage
curl	Tests manuels rapides

tests/integration.sh	Suite automatisée 57 scénarios
Postman	Collection générable depuis api-documentation.md

11.10 Code back — Récurrence

Logique métier intéressante : avancement automatique des tâches récurrentes.

```
// back/src/models/task.model.js
async advanceRecurring(id, completedBy, frequency) {
  const nMap = { daily: '1', weekly: '7', monthly: '1', yearly: '1' };
  const unit = (frequency === 'daily' || frequency === 'weekly') ? 'DAY'
    : (frequency === 'monthly') ? 'MONTH'
    : 'YEAR';
  await pool.query(
    `UPDATE tasks
      SET due_date = DATE_ADD(due_date, INTERVAL ${nMap[frequency]} ${unit}),
          completed_at = NOW(),
          completed_by = ?
      WHERE id = ?`,
    [completedBy, id]
  );
}
```

Note : `frequency` est validé en amont via énumération whitelisted → interpolation `${...}` sûre.

11.11 Mini-bilan Mission 2

✓ Toutes les user stories tâches + validation parent sont livrées et testées (15/57 scénarios concernent cette mission).

Le flow `pending_review` a nécessité une migration (`005_pending_review.sql`) pour étendre l'énumération `status`.

12. Mission n°3 — Export iCal (intégration standard tiers)

12.1 Introduction

Cette mission est le **défi technique central** du projet : permettre aux utilisateurs de recevoir des **notifications natives sur leur téléphone** (iPhone et Android) sans installer aucune application.

12.2 Le problème

Les approches classiques pour notifier un téléphone :

Approche	Problème
Application native (Swift/Kotlin)	2 stores, 2 codebases, comptes développeur payants, mise à jour, support
Web Push API	Limité sur iOS (PWA installée obligatoire)

Email	Nécessite SMTP, configuration domaine, anti-spam
SMS	Coûteux, dépend d'un service tiers (Twilio...)

12.3 La solution : iCalendar (RFC 5545)

iCalendar est un standard ouvert de 1998 que **tous les calendriers natifs comprennent** : iPhone Calendar, Google Calendar, Outlook, Thunderbird, etc.

En exposant un flux `.ics` accessible par URL, on permet aux applications calendrier de :

1. S'abonner au flux (1 seule fois)
2. Poller périodiquement la mise à jour
3. Déclencher des **VALARM** = notifications système (lock screen)

12.4 Diagramme de séquence

Voir [diagrammes/sequence-ical.mmd](#) .

12.5 Bibliothèque utilisée : ical-generator

Service tiers npm qui génère du RFC 5545 valide depuis des objets JS.

```
npm install ical-generator
```

12.6 Code back — Service de génération

```
// back/src/services/ical.service.js
import ical from 'ical-generator';

export function buildIcal({ ownerName, calendarName, calendarColor, tasks }) {
  const cal = ical({
    name: calendarName || `Reminder ${ownerName}`,
    prodId: { company: 'Reminder Famille', product: 'Reminder', language: 'FR' },
    timezone: 'Europe/Paris',
  });

  if (calendarColor) cal.x('X-APPLE-CALENDAR-COLOR', calendarColor);

  tasks.forEach((task) => {
    const event = cal.createEvent({
      id: `task-${task.id}@reminder-famille`,
      start: new Date(task.due_date),
      allDay: true,
      summary: task.title,
      description: task.description || '',
      categories: [{ name: task.category }],
    });

    // VALARM : notification système 1 jour avant
    event.createAlarm({
      type: 'display',
```

```

    trigger: 86400, // 1 day in seconds (négatif = avant)
    description: task.title,
  });
});

return cal.toString();
}

```

12.7 Endpoint et sécurité par token

```

// back/src/routes/calendar.routes.js
router.get('/:token', asyncHandler(calendarController.export));
router.get('/:token/perso.ics', asyncHandler(calendarController.exportPersonal));
router.get('/:token/family/:familyId.ics',
  asyncHandler(calendarController.exportFamily));

```

Authentification spéciale : pas de JWT (impossible avec une URL abonnée par un calendrier externe). Le `calendar_token` (48 caractères hex aléatoires) sert d'authentification dans l'URL.

Sécurité acceptable :

- Entropie 192 bits → brute-force inenvisageable
- Accès **read-only** : aucune mutation possible
- L'utilisateur peut régénérer son token (futur)

12.8 Code front — QR code

```

// front/src/pages/Profile.jsx (extrait)
import QRCode from 'qrcode';

useEffect(() => {
  const url = `webcal://${window.location.host.replace(':5173', ':3000')} +
    `/api/calendar/${user.calendar_token}`;
  QRCode.toCanvas(canvasRef.current, url, { width: 200 });
}, [user.calendar_token]);

```

Le préfixe `webcal://` au lieu de `http://` est compris par iOS et Android comme « ouvrir dans l'application Calendrier » directement.

12.9 Screenshots résultat

Voir `annexes/screenshots/09-profile-ical.png` dans le repo projet — montre le QR code généré et le bouton « Ouvrir dans Calendrier ».

12.10 Mini-bilan Mission 3

- ✅ Les 3 flux iCal sont fonctionnels (global, perso, par famille).
- ✅ Testés sur iPhone (iOS 17) et Android (Pixel 6) : abonnement réussi, notifications déclenchées à H-24.

Difficultés :

- Choix entre `http://` (téléchargement seul) et `webcal://` (abonnement direct) — préférence pour `webcal://` car bien plus user-friendly
- Couleur du calendrier non-standard — utilisation de `X-APPLE-CALENDAR-COLOR` (extension Apple, ignorée par Google sans nuire)

13. Déploiement

13.1 État actuel

Le projet est **fonctionnel en local uniquement**. Le déploiement a été priorisé après le polish fonctionnel (choix assumé pour soutenance).

13.2 Plan de déploiement (préparé)

Composant	Cible	Coût
Front (build Vite)	Vercel	gratuit
Back (Express)	Fly.io	gratuit (3 machines partagées)
BDD (MariaDB)	Fly.io machine séparée + volume persistant	gratuit

13.3 Bonnes pratiques générales

Gestion du `.env`

- Versionné : `.env.example` avec valeurs factices documentant les variables
- Non versionné : `.env` réel (gitignored)
- En production : variables injectées via `fly secrets set DB_PASSWORD=...`

Séparation des environnements

- `development` : local, MariaDB local, logs verbose
- `staging` : Fly.io app dédiée, BDD séparée, données de test
- `production` : Fly.io app prod, BDD prod, monitoring

13.4 Conteneurisation (Docker — futur)

```
# back/Dockerfile (prévu)
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY src/ ./src/
COPY migrations/ ./migrations/
EXPOSE 3000
CMD ["node", "src/index.js"]
```

13.5 Scripts de déploiement

- `back/package.json` :

```
"scripts": {
  "migrate": "node src/db/migrate.js",
  "seed":    "node src/db/seed.js",
  "start":   "node src/index.js"
}
```

13.6 Tests d'intégration

Suite de 57 scénarios dans `tests/integration.sh` :

- Authentification (11 tests)
- Familles (14 tests)
- Tâches CRUD et permissions (11 tests)
- Validation parent (10 tests)
- Export iCal (7 tests)
- Statistiques (2 tests)
- Validation entrée invalide (transversal)

```
$ bash tests/integration.sh
```

Résultats : 57 passés / 57 total
Tous les tests passent !

14. Sécurité

Voir [securite.md](#) — document complet OWASP avec extraits de code défensif.

Récapitulatif :

Risque OWASP	Mesure
Injection SQL	100% requêtes préparées
XSS	React échappement automatique
CSRF	JWT en header (pas de cookie)
Auth brute force	Rate limit + message générique
Cryptographic Failures	bcrypt cost 10, JWT HS256
Broken Access Control	Middlewares composables, last-admin protection
Security Misconfig	CORS restreint, NODE_ENV strict
Logging	Stack traces masquées au client

15. Conclusion générale

15.1 Bilan du travail réalisé

Fonctionnalités livrées vs prévues

Prévu CDC v4	Livré
Authentification (register, login, profil)	✓
Familles (création, jointure, validation)	✓
Tâches CRUD + récurrence	✓
Validation parent (pending_review)	✓
Export iCal global	✓
Dashboard adapté par rôle	✓
Statistiques	✓ (Chart.js — au-delà du CDC)
Calendrier mensuel	✓ (au-delà du CDC)
Commentaires sur tâches	✓ (au-delà du CDC)
Sous-feeds iCal	✓ (au-delà du CDC)
PWA installable	✓ (au-delà du CDC)
Mode sombre	✓ (au-delà du CDC)
Reset MDP par admin	✓ (au-delà du CDC)
Déploiement en ligne	⏸ Reporté

Résultat : 100% du CDC livré + 8 fonctionnalités au-delà.

Analyse des erreurs et corrections

Difficulté	Solution
Pivot mid-projet (perso → familles)	Refactoring complet schéma BDD, ~3 jours
Last-admin logout possible	Ajout vérification <code>countAdmins()</code> à 2 endroits
<code>updateMember</code> acceptait tout body	Refactoring en whitelist explicite <code>{role, is_admin}</code>
iCal cache navigateur	Header <code>Cache-Control: no-cache</code> ajouté
Tests manuels longs et oubliés	Création de la suite <code>integration.sh</code> (57 scénarios)

15.2 Bilan de la formation

Cette section nécessite une réflexion personnelle du candidat. Points à aborder :

- Retour sur la reconversion (parcours initial → DWWM)
- Difficultés rencontrées pendant la formation et comment elles ont été surmontées
- Compétences nouvellement acquises (back-end, front-end, BDD, DevOps)
- Apprentissages transversaux (gestion de projet, autonomie, recherche)

15.3 La suite

Évolutions techniques identifiées

- **Tests unitaires** (Vitest) en complément de l'intégration
- **CI/CD** GitHub Actions (lint + tests + déploiement auto)
- **Notifications email** via Resend ou Brevo (SMTP)
- **Bot Telegram** pour rappels temps réel additionnels
- **PWA mode offline** avec service worker
- **TypeScript** pour catch d'erreurs au build
- **Internationalisation** (anglais en plus du français)

Perspectives personnelles

Cette section nécessite une réflexion personnelle :

- *Poursuite d'études envisagée (Bachelor, Master) ?*
 - *Recherche d'emploi (poste visé, entreprise type) ?*
 - *Projet entrepreneurial ?*
-

Annexes

Annexes placées en dernière partie du dossier.

- A1 — Cahier des charges détaillé ([CAHIER DES CHARGES.md](#))
- A2 — Diagrammes Mermaid sources ([diagrammes/](#))
- A3 — Documentation API complète ([api-documentation.md](#))
- A4 — Document sécurité OWASP ([securite.md](#))
- A5 — User stories formalisées ([user-stories.md](#))
- A6 — Charte graphique ([maquettes/charte-graphique.md](#))
- A7 — Wireframes basse fidélité ([maquettes/wireframes.md](#))
- A8 — 17 screenshots haute fidélité ([annexes/screenshots/](#))
- A9 — Suite de tests d'intégration (57 scénarios) ([tests/integration.sh](#))
- A10 — Plan de soutenance orale ([SOUTENANCE.md](#))